ELDRITCH ARENA / APLICAÇÃO PRÁTICA DE FRAMEWORKS

Ficha de planejamento

Instituição: UNINORTE | Curso: Sistemas de Informação

Disciplina: Padrões de Projeto e Frameworks | Professor: Bruno Rezende

Aluno: João Carlos Campos | Projeto: Eldritch Arena

Problema e público

Jogadores precisam encontrar eventos e parceiros de card games. Lojistas e organizadores precisam reunir inscrições, anúncios e quantidades de cartas com histórico confiável. A proposta atende jogadores, lojistas e organizadores. Esse contexto orienta o projeto, sem alegar pesquisa de campo ou resultados medidos.

Objetivo

Desenvolver uma plataforma web que reúna torneios, comunidade, ferramentas de mesa e controle de estoque. No fluxo principal, o lojista deve cadastrar uma carta, registrar entradas e saídas e consultar o saldo e seu histórico, enquanto o sistema impede estoque negativo e acesso de outra loja.

Solução e escopo

Aplicação Laravel 13 em PHP 8.4, com PostgreSQL 18, páginas Blade, Tailwind CSS e Alpine.js. Inclui autenticação, eventos, anúncios, comunidade, contador de vida, esquemas de Archenemy, estoque e consulta administrativa. Pagamentos, entrega, reserva e baixa automática por venda ficam fora desta etapa.

Framework e justificativa

Laravel organiza rotas, middleware, validação, ORM Eloquent, migrations e views Blade. Esses recursos se ajustam à base existente em PHP e reduzem o trabalho de infraestrutura para um desenvolvimento individual. O projeto continua responsável por implementar suas regras específicas de propriedade, saldo e publicação.

Arquitetura e protótipos

Arquitetura MVC, complementada por um serviço de estoque. As páginas seguintes apresentam requisitos, diagrama e três wireframes: login, painel da loja e cadastro de carta.

Enquadramento da atividade

O sistema já está em desenvolvimento. Esta ficha organiza seu planejamento com base no código atual. Os wireframes são reconstruções didáticas das telas, e não evidência de uma etapa anterior à programação. A realização individual e o tema próprio dependem de alinhamento com o professor, pois o enunciado prevê grupos de 3 a 5 alunos.

Requisitos e aceitação

Requisitos funcionais

RF01 (Alta) — Cadastrar usuários, autenticar e encerrar sessão. Rotas privadas exigem login.

RF02 (Alta) — Criar e gerenciar torneios e inscrições. Recusar inscrição sem vaga disponível.

RF03 (Média) — Cadastrar e consultar anúncios. Ocultar da vitrine o item vinculado sem saldo.

RF04 (Média) — Criar tópicos, comentários e reações. Restringir edição/exclusão ao acesso autorizado.

RF05 (Média) — Escolher jogo/formato e controlar vida, marcadores, dados e esquemas. Preservar a mesa no navegador.

RF06 (Alta) — Cadastrar cartas com SKU, jogo, condição, custo, preço e saldo inicial. SKU único por loja.

RF07 (Alta) — Registrar entradas/saídas com motivo, autor e saldo. Exibir estoque mínimo e impedir saldo negativo.

RF08 (Média) — Permitir ao administrador consultar contas e indicadores. Recusar acesso sem privilégio.

Requisitos não funcionais

RNF01 — Segurança: Usar hash de senha, sessão, CSRF e autorização por perfil/proprietário. Verificar recusa de acesso de outra loja.

RNF02 — Integridade: Gravar saldo e histórico na mesma transação. Reenvio do identificador da operação não deve duplicar o movimento.

RNF03 — Usabilidade: Meta: validar formulários em 390 e 1366 px, sem cortes nos controles essenciais, com rótulos e erros legíveis.

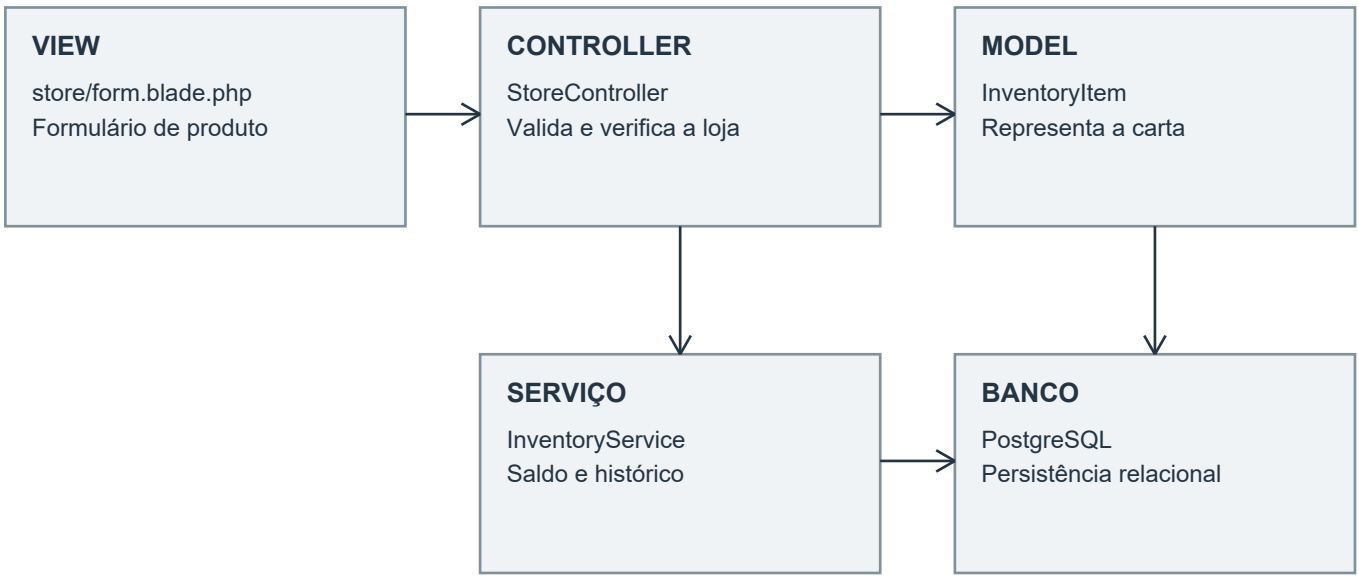
RNF04 — Desempenho: Meta ainda não medida: 95% das consultas em até 2 s, com 10 usuários simultâneos e mil produtos por loja, em ambiente de teste definido.

RNF05 — Manutenção: Separar MVC/serviços, versionar migrations e manter testes de permissão, saldo e publicação antes das entregas.

RNF06 — Recuperação: Fazer backup antes de publicar. Meta: ensaiar restauração em ambiente isolado e comparar registros. Não há SLA de disponibilidade comprovado.

Arquitetura MVC aplicada

Exemplo: cadastrar uma carta e registrar seu saldo inicial. O serviço concentra regras que precisam de transação e podem ser reutilizadas.



Sequência da operação

1. O lojista preenche o formulário e envia POST /loja/produtos.
2. Middleware e controller verificam sessão, permissão e loja proprietária.
3. O controller valida os campos e cria InventoryItem pelo Eloquent.
4. Se houver saldo inicial, InventoryService registra StockMovement e atualiza o saldo na transação.
5. O controller redireciona para os detalhes e apresenta a confirmação.

Entidades e relações

User possui uma Store. Store possui vários InventoryItem. Cada item tem vários StockMovement e pode ter um CardListing vinculado. A movimentação também registra o usuário responsável. Admin é um privilégio separado do perfil jogador/organizador.

Caso de teste demonstrável

Cadastrar 5 unidades, retirar 2 e conferir saldo 3. Tentar retirar 4 deve falhar. Reenviar a mesma retirada não deve gerar outra baixa. Executar essa demonstração em dados de teste, sem alterar estoque real.

Protótipos: login e painel

Wireframes simplificados, baseados nas views existentes. Campos, botões e navegação estão identificados. Dados de produtos e saldos são fictícios.

TELA 1 — ENTRAR NA ARENA

E-mail

Senha

[Lembrar acesso] [Entrar] Cadastrar

Entrar valida as credenciais e abre o painel. Cadastrar abre o registro. Erros aparecem junto ao formulário, sem exibir a senha.

TELA 2 — MINHA LOJA

Menu: Torneios | Marketplace | Comunidade | Minha loja

Resumo: 1 produto / 5 unidades / 0 itens com estoque baixo

Busca: [nome ou SKU] Filtro: [todos]

Ação principal: [Cadastrar produto]

EX-001 | Carta exemplo | Saldo: 5 | [Abrir item]

Abrir item permite consultar histórico e registrar movimentos. O menu dá acesso aos demais módulos. Somente a loja proprietária acessa seus produtos.

Protótipo: cadastro e conclusão

TELA 3 — CADASTRAR PRODUTO

SKU / código interno	**Nome da carta**
Jogo / edição	**Raridade / condição**
Custo / preço de venda	**Quantidade inicial / mínimo**

Descrição pública / URL da imagem (opcionais)

[Salvar produto] [Voltar ao estoque]

Salvar valida os dados e cria o produto privado. O lojista publica depois, pela tela de detalhes. A quantidade inicial aparece no cadastro; alterações posteriores de saldo exigem movimentação com motivo. Campos inválidos recebem mensagens de erro.

Conclusão

O Eldritch Arena reúne a participação em eventos e o apoio às partidas com o controle de estoque das lojas. O Laravel fornece uma estrutura MVC reutilizável, enquanto as regras do projeto garantem propriedade dos dados e rastreabilidade dos saldos. As próximas validações incluem uso em celular, desempenho e recuperação de backup.

Fontes utilizadas

Material da disciplina: Aplicação Prática de Frameworks, UNINORTE, Prof. Bruno Rezende, slides 5, 9–10 e 18–19. Repositório Eldritch Arena, branch feature/arena-professional, referência 58dba21: composer.json, routes/web.php, StoreController.php, InventoryService.php, views de login/estoque e testes de estoque.

https://github.com/JCUNME12/eldritch-arena/tree/feature/arena-professional

Este documento organiza a atividade de planejamento. Recursos já implementados não comprovam, por si só, as metas de desempenho ou disponibilidade propostas.